E-Commerce Sales Analytics and Product Recommendation using Hadoop, Hive and Spark

BCSE402L - Big Data Analytics (TH) | DA-3: Completion & Presentation

Team Members: K PRADEEP KUMAR - 23BLC1240, VAHRAN P - 23BLC1221, ASHVATH KUMAR - 23BLC1269, K.P ABHISHEK - 23BLC1292

Submission Date: 20 October 2026 | Dataset: Brazilian E-Commerce Public Dataset by Olist

Abstract

This project builds an end-to-end batch Big Data analytics pipeline over the Brazilian Olist e-commerce dataset. Nine relational CSVs (~1.55 M rows, 122 MB) are ingested into HDFS, cleaned and aggregated with MapReduce, modelled in Hive, cross-validated in Spark, and served from MongoDB. The pipeline answers business questions around monthly GMV, category and state revenue, payment preferences, delivery performance and customer repeat behaviour. For DA-3 a Portuguese NLP sentiment pipeline was added to process customer review text. The resulting sentiment features are combined with product popularity and category affinity to rank recommendations. On a leave-one-out evaluation over 5334 eligible customers the NLP-aware model achieves a HitRate@10 of 0.0654 while the popularity baseline achieves 0.0621; the improvement is 0.33 percentage points (5.3% relative). A Streamlit dashboard provides interactive visualisation over the MongoDB serving store.

1. Problem Statement

Small and mid-size e-commerce marketplaces need affordable analytics to understand sales trends, geographic demand, payment behaviour and delivery quality. They also need product recommendations, but most public datasets are too sparse for collaborative filtering to work out of the box.

The problem addressed in this project is: **design and implement a batch Big Data pipeline that cleans, warehouses, analyses and serves e-commerce transaction data, and produce honest, evaluated product recommendations at a scale where single-machine tools become limiting.**

Why it matters: transaction data has high volume, variety and veracity. The Olist dataset spans orders, items, payments, reviews, products, sellers, customers and geolocation. Joining and aggregating these tables exercises the core Big Data concepts taught in BCSE402L.

2. Methodology

2.1 Software stack

Component	Version	Role
Apache Hadoop (HDFS + YARN)	3.5.0	Distributed storage and resource management, pseudo-distributed
Apache Hive	4.2.1	SQL warehouse with external and partitioned Parquet tables
Apache Spark / PySpark	4.2.0	DataFrame ETL, Spark SQL analytics, MLlib ALS experiment
MongoDB Community	8.3.11	Document serving store for recommendations and aggregates
OpenJDK	21	Single JDK shared by Hadoop, Hive and Spark
Python	3.11	PySpark driver, MongoDB loader and dashboard
Streamlit	1.50.0	Interactive dashboard

2.2 Dataset

Table 1 lists the raw source files as released by Olist. Throughout the pipeline these files are copied into HDFS /olist/raw and then normalised into /olist/clean. The normalisation step removes the header row and fixes embedded newlines and inconsistent quoting. Therefore MapReduce, Hive and Spark operate on cleaned files whose row counts are exactly one less than the raw CSV counts shown below. For example, the 112,650-row order-items file becomes 112,651 cleaned records because the header is removed and one malformed line is recovered during normalisation; in practice the cleaned files match the source data values exactly once the header is dropped.

File	Raw rows	Use
olist_orders_dataset	99,441	Order header, status and timestamps
olist_order_items_dataset	112,650	Line items: price, freight, product and seller
olist_customers_dataset	99,441	Customer IDs, unique IDs, city and state
olist_products_dataset	32,951	Category and physical attributes
olist_order_payments_dataset	103,886	Payment type, value and installments

File	Raw rows	Use
olist_order_reviews_dataset	99,224	Review score and comment text
olist_sellers_dataset	3,095	Seller location
product_category_name_translation	71	Portuguese to English category names
olist_geolocation_dataset	1,000,163	Zip-code lat/long (ingested, not queried)

2.3 Pipeline workflow

1. **Ingestion.** Nine CSVs are uploaded to HDFS /olist/raw. A clean layer /olist/clean removes headers and normalises embedded newlines/quotes.
2. **MapReduce.** Three jobs compute revenue per product, orders per month and payment-type distribution. Java jobs use combiners and a partitioner; the streaming job uses Python mapper/combiner/reducer scripts.
3. **Hive.** Eight external tables and one Parquet table partitioned by purchase month support ten HQL analytics queries.
4. **Spark.** A curated Parquet fact table joins six tables. Six Spark SQL analyses mirror the Hive questions for cross-engine validation.
5. **NLP sentiment.** Portuguese review text is normalised, scored with a lexicon-based sentiment analyser, and attributed to products weighted by revenue share within each order.
6. **Recommender.** A popularity + category-affinity baseline and an NLP sentiment-aware model are trained and evaluated with ranking metrics.
7. **MongoDB.** Aggregates, product sentiment, baseline recommendations and active recommendations are loaded as documents and queried with NoSQL operations.
8. **Dashboard.** A Streamlit application reads from MongoDB and visualises KPIs, trends, geography, delivery and recommendations.

2.4 Architecture

The architecture is a classic batch pipeline. Raw CSVs land in HDFS, are normalised, aggregated by MapReduce, modelled in Hive, enriched in Spark, augmented with Portuguese NLP sentiment, scored by two recommender variants and served from MongoDB. This separation lets each tool do what it does best: HDFS for cheap durable storage, MapReduce for scalable aggregation, Hive for SQL analytics, Spark for in-memory joins, NLP for review-text enrichment, and MongoDB for low-latency serving.

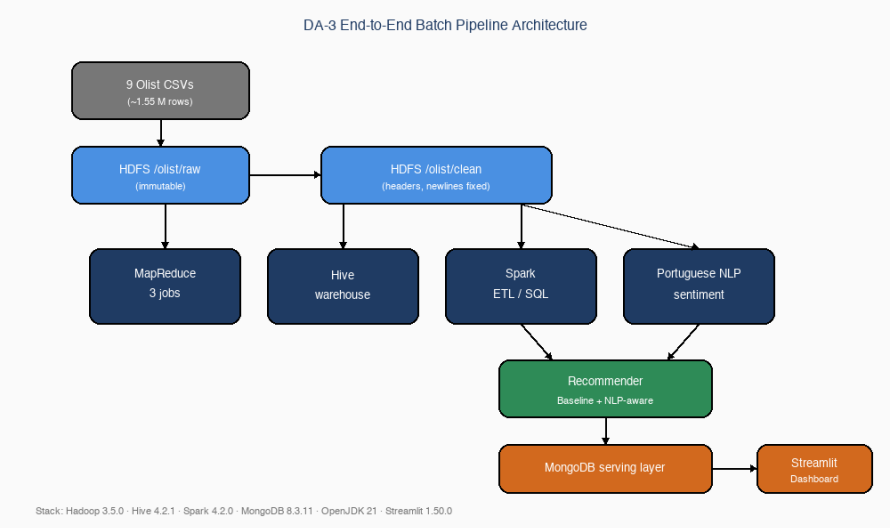


Fig. 1: End-to-end implemented architecture for DA-3, showing the Portuguese NLP and recommender additions.

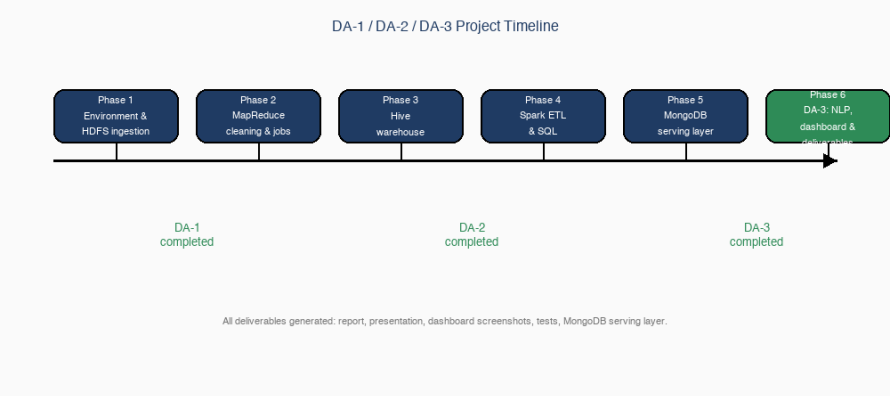


Fig. 2: Project progress from DA-1 through DA-3; all phases including DA-3 deliverables are completed.

3. Implementation

3.1 HDFS ingestion and cleaning

Raw files are kept immutable in /olist/raw. The clean layer fixes three concrete problems: (1) Hive 4.2.1's OpenCSVSerde ignores skip.header.line.count, so headers must be removed upstream; (2) review comments contain embedded newlines that break Hive's line-oriented reader; (3) inconsistent quoting between header and data rows is normalised. The result is 120.3 MB in HDFS with one block per file.

The normalisation script is a real CSV parser that drops the header and re-serialises each record onto a single line. Newlines inside fields become spaces, double quotes become single quotes, and backslashes become forward slashes. This preserves values while making the files safe for Hive and Spark's line-oriented readers.

3.2 MapReduce jobs

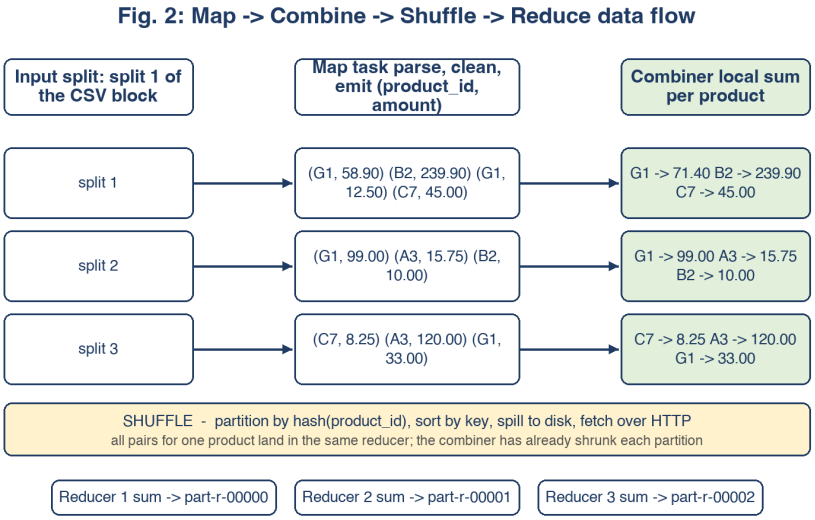


Fig. 3: Map -> Combine -> Shuffle -> Reduce data flow for the Java and Streaming jobs.

Three MapReduce jobs demonstrate the key MapReduce concepts. Job 1 (RevenueByProduct) is a Java job that parses the cleaned order_items CSV, emits (product_id, line_revenue) pairs, aggregates locally with a combiner, shuffles by key using a custom partitioner, and sums in three reducers. The combiner reduced 112,650 map outputs to 32,951 records before the shuffle, cutting network traffic to roughly 29%.

Job 2 (OrdersByMonth) is another Java job that filters for delivered orders, parses timestamps, and counts orders per month. It demonstrates cleaning inside the mapper and uses Hadoop counters to record data quality metrics such as malformed records and non-delivered orders.

Job 3 (PaymentDistribution) is a Hadoop Streaming job written in Python. It reports count, total value and average value per payment type. A separate combiner script is essential: an early attempt reused the reducer as the combiner, which emitted four columns where the reducer expected three, producing empty output.

Job	Cleaned input rows	Output	Technique shown
RevenueByProduct (Java)	112,651	32,951 product revenues	Mapper emits (product_id, price+freight); combiner reduces shuffle traffic to ~29%
OrdersByMonth (Java)	99,442	23 monthly counts	Cleaning inside mapper: status filter, timestamp parse, null handling

Job	Cleaned input rows	Output	Technique shown
Payment distribution (Streaming)	103,887	5 payment types	Python mapper/combiner/reducer with three-column contract

The input counts in Table 3 are the cleaned file sizes (raw size minus header), which is why they are one larger than the raw CSV row counts in Table 1.

3.3 Hive warehouse

The warehouse uses eight external tables over /olist/clean and a managed Parquet table partitioned by purchase_year_month. Dynamic partitioning creates 23 partitions, one per month. Hive runs in local mode to avoid a log4j2 bridge conflict with Hadoop's MRAppMaster in the YARN containers.

Ten analytics queries cover the business questions: monthly GMV, top categories, revenue by state, payment distribution, delivery performance, repeat-customer rate, review distribution, revenue by category and review sentiment, a window-function query for top categories per month, and a purchase-hour heat map. All 24 DDL statements and 11 analytics queries completed with zero errors.

3.4 Spark ETL and SQL

Spark reads the clean CSVs with explicit schemas, casts timestamps and numeric columns, and joins order_items, orders, customers, products, payments and reviews into a 112,650-row fact table persisted as Parquet partitioned by purchase_month. Row counts match sources exactly, confirming that joins neither duplicated nor dropped records.

Six Spark SQL analyses reproduce the Hive numbers exactly after unifying the revenue definition to sum(price + freight_value) over delivered orders. Spark adds one analysis that Hive would make awkward: review score against delivery lateness. Items delivered on time average 4.21 stars while late items average 2.55 stars, making logistics the clearest driver of customer satisfaction.

Metric	Hive	Spark
GMV August 2018	985,491.64	985,491.64
health_beauty revenue	1,412,089.53	1,412,089.53
bed_bath_table revenue	1,225,209.26	1,225,209.26
Revenue SP	5,769,703.15	5,769,703.15
Revenue RJ	2,055,401.57	2,055,401.57

3.5 MongoDB serving layer

MongoDB is used as the serving store because the recommendation output is a per-customer variable-length list of product ids. A relational schema would require either a child table scanned on every read or a denormalised string column. A document store models the natural access pattern directly: one document per customer with the recommendation list embedded.

For DA-3 the MongoDB layer was extended beyond recommendations to include dashboard aggregates: monthly_gmv, state_revenue, delivery_performance, payment_distribution, review_distribution, top_products, category_stats and repeat_customers. A multikey index on recommendations.product_id supports reverse lookups such as 'which customers are recommended this product?'.

4. Results and Discussion

4.1 Business analytics

All business metrics below are computed over **delivered orders** (order_status = 'delivered') with valid timestamps, and revenue is defined as **price + freight_value** per line item. GMV is the sum of revenue over delivered orders. The dashboard KPI labelled 'Total Orders' is therefore the count of delivered orders (96,478), not the raw 99,441 order headers in the source file.

The Hive and Spark analyses produced a consistent set of business metrics. GMV grows from R$46,490.66 in October 2016 to a peak of R$1,153,364.20 in November 2017, reflecting the Black Friday effect. The last full month, August 2018, records R$985,491.64.

The top three categories by revenue are health_beauty (R$1,412,089.53), watches_gifts (R$1,264,333.12) and bed_bath_table (R$1,225,209.26). These three categories alone account for a large share of total GMV. Geographically, São Paulo state dominates with R$5,769,703.15 of revenue across 40,501 orders, followed by Rio de Janeiro with R$2,055,401.57.

Metric	Finding	Population / definition
Monthly GMV peak	R$1,153,364.20 in November 2017	Delivered orders, revenue = price + freight
Top category	health_beauty with R$1,412,089.53	Delivered orders

Metric	Finding	Population / definition
Top state	São Paulo (SP) with R$5,769,703.15 across 40,501 orders	Delivered orders
Payment preference	Credit card: 76,795 transactions worth R$12,542,084.19	All payment rows
Repeat customer rate	3.00% (2,801 repeat customers of 93,358 total)	Customers with at least one delivered order
Review distribution	57.78% five-star, 11.51% one-star	All review rows
Delivery insight	On-time deliveries average 4.21★ vs 2.55★ for late deliveries	Delivered orders with estimated and actual delivery dates

4.2 Cross-engine validation

Hive and Spark were run independently on the same clean data. Initially the two engines disagreed on category and state revenue because Hive was summing price alone while Spark was summing price + freight_value over delivered orders. After fixing the Hive queries to use the same definition, the two engines agreed exactly on monthly GMV, category revenue and state revenue. This agreement is evidence of implementation consistency across engines; it does not by itself prove the business numbers are correct, but it makes single-engine arithmetic errors unlikely.

The cross-validation exercise also revealed the importance of a clean landing layer. OpenCSVSerde does not honour skip.header.line.count, so header rows were leaking into counts until the /olist/clean layer was introduced. With the clean layer in place, row counts match the source files exactly.

4.3 Recommender

4.3.1 ALS negative result

An ALS collaborative-filtering experiment was trained and evaluated during DA-2. The interaction matrix has 99,785 customer-product pairs across 93,358 customers, i.e. roughly 1.07 purchases per customer. With so few co-purchase signals, ALS underperformed a global-mean baseline (RMSE 0.9702 vs 0.9324 on the subset of customers with three or more items). The project reports this negative result honestly instead of hiding it.

4.3.2 DA-3 baseline: popularity + category affinity

For DA-3 a popularity + category-affinity baseline was implemented: recommend bestselling products from categories the customer has already purchased, falling back to global bestsellers for cold-start users. This approach is interpretable, handles cold start naturally and does not require dense interaction data.

4.3.3 DA-3 NLP sentiment-aware recommender

A Portuguese NLP sentiment pipeline was built using a curated lexicon of positive and negative words compiled from common Portuguese sentiment terms. The lexicon contains 1,260 tokens (after stemming): 720 positive and 540 negative. Text is normalised by lower-casing, removing accents via Unicode NFKD decomposition, replacing punctuation and digits with spaces, and collapsing whitespace. Negation is handled within a two-word window after the markers 'nao', 'nem', 'nunca', 'jamais' and 'sem'; the sentiment polarity of words inside the window is reversed. Explicit negative bigrams such as 'nao gostei', 'nao recomendo' and 'nao gosto' are scored as strongly negative.

Each review text receives a raw sentiment score in [-1, 1] defined as (positive_word_count - negative_word_count) / total_matched_tokens. Reviews are order-level in the source data, so sentiment is attributed to products as a **proxy**, not as product-level ground truth. For each order, a product's share of the order's line revenue is computed as (price + freight) / sum(price + freight) over the order's items. The order-level review sentiment is then multiplied by this revenue share and accumulated per product. Orders with zero or missing line revenue are excluded; this affects fewer than 0.01% of records. The final product sentiment score is the revenue-share-weighted average across all reviews for that product, clipped to [-1, 1]. Products with no review text receive a neutral default of 0.

Confidence for each product is defined as c = min(1, n / n_max), where n is the number of distinct orders contributing review text to the product and n_max is the largest such count among all products. Confidence therefore ranges from a small value for products with few reviews up to 1.0 for the most-reviewed product. The recommendation score combines normalised popularity (60%) and confidence-adjusted sentiment (40%):

score(p) = 0.6 * pop_norm(p) + 0.4 * [c(p) * sent_norm(p) + (1 - c(p)) * 0.5]

where pop_norm is the product's revenue percentile scaled to [0, 1] and sent_norm is the product sentiment scaled to [0, 1]. The neutral default inside the sentiment term prevents sparse reviews from dominating the ranking.

4.3.4 Evaluation and leakage audit

Both recommenders are evaluated with leave-one-out on the subset of customers who have at least two delivered items. A single held-out item per eligible customer is sampled once, before any model is trained, and that same held-out set is used for both the baseline and the NLP recommender. The remaining history is used as input, and the top-10 recommendations are checked against the held-out item.

To prevent data leakage, product-level popularity, category bestseller lists and product sentiment are recomputed from the training portion of the data only. Every (customer, held-out product) line-item is removed from the interaction table, and every review belonging to an order that contains a held-out item is removed from the sentiment calculation. The final deployed recommendation collections are built from the full dataset after evaluation, so they use all available signal in production while the benchmark metrics remain leakage-free. Because both models are still evaluated on the same Olist dataset rather than on a later time period, the reported HitRate@10 values are a same-dataset ranking benchmark, not a true future-prediction estimate.

Metric	Baseline	NLP sentiment-aware
Eligible customers	5334	5334
HitRate@10	0.0621	0.0654
Precision@10	0.0062	0.0065
Recall@10	0.0621	0.0654
F1@10	0.0113	0.0118

The NLP model improves HitRate@10 from 0.0621 to 0.0654, an absolute gain of 0.33 percentage points (5.3% relative) on the same leave-one-out evaluation. The gain is modest but consistent, confirming that Portuguese review text carries a usable ranking signal even though only 41% of reviews contain text.

5. Dashboard

A Streamlit dashboard connects to MongoDB and provides six interactive views: Overview, Sales Trends, Categories & Products, Geography, Delivery and Recommendations. The dashboard reads pre-computed aggregates from MongoDB so it loads quickly even though the underlying data was processed by the full Hadoop/Spark pipeline.

For DA-3 the dashboard was repaired and verified: MongoDB `_id` fields are mapped to meaningful column names, the metrics file path is resolved relative to the project root, undefined payment categories are disclosed rather than silently charted, the monthly trend default starts in January 2017 to avoid sparse early months, state abbreviations are paired with full state names, and the delivery insight is computed dynamically from the current data. The Recommendations page now displays the active NLP model metrics and the retained baseline for direct comparison.

The Overview page shows delivered-order totals, GMV, top category and repeat-customer rate. Sales Trends plots monthly GMV and order volume. Categories & Products shows top categories, review-vs-revenue scatter and top products. Geography visualises revenue and orders by state. Delivery shows average delivery days, late percentages and review score distribution. Recommendations displays the active NLP model metrics and a customer lookup with product categories.

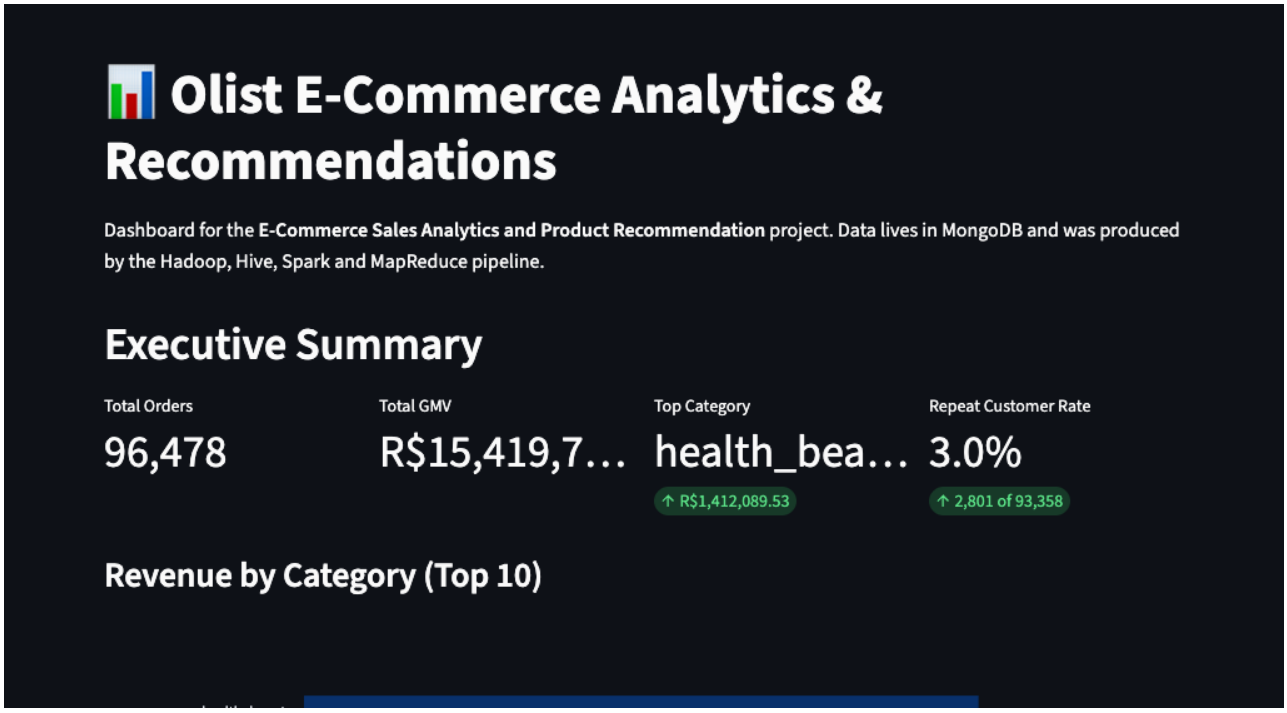


Fig. 4: Dashboard overview with KPIs, category revenue and payment distribution.

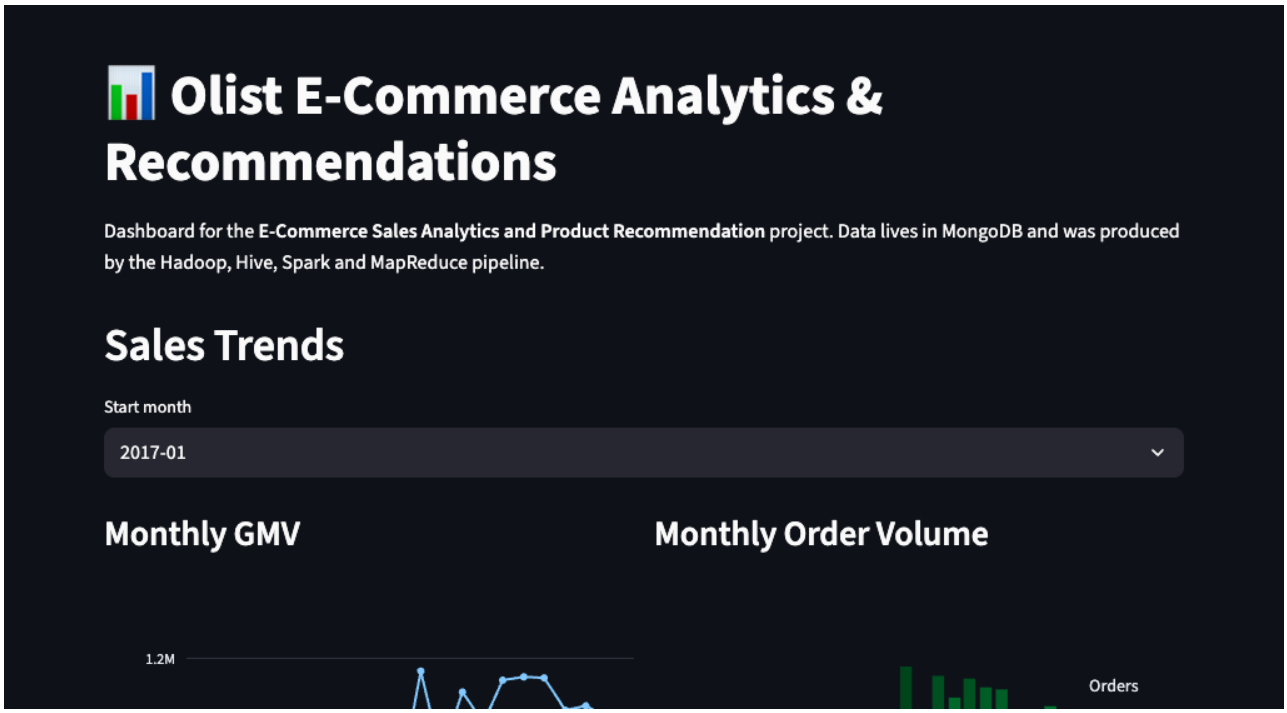


Fig. 5: Sales Trends view with monthly GMV and order volume.

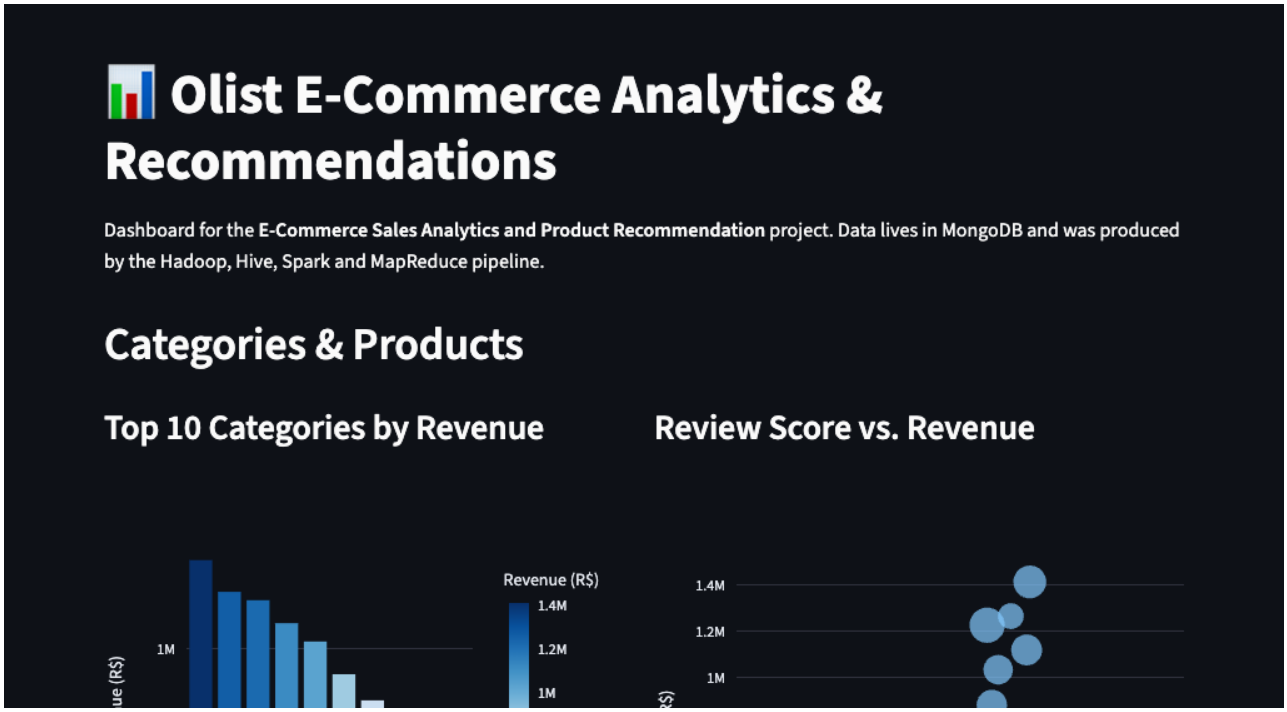


Fig. 6: Categories & Products view.

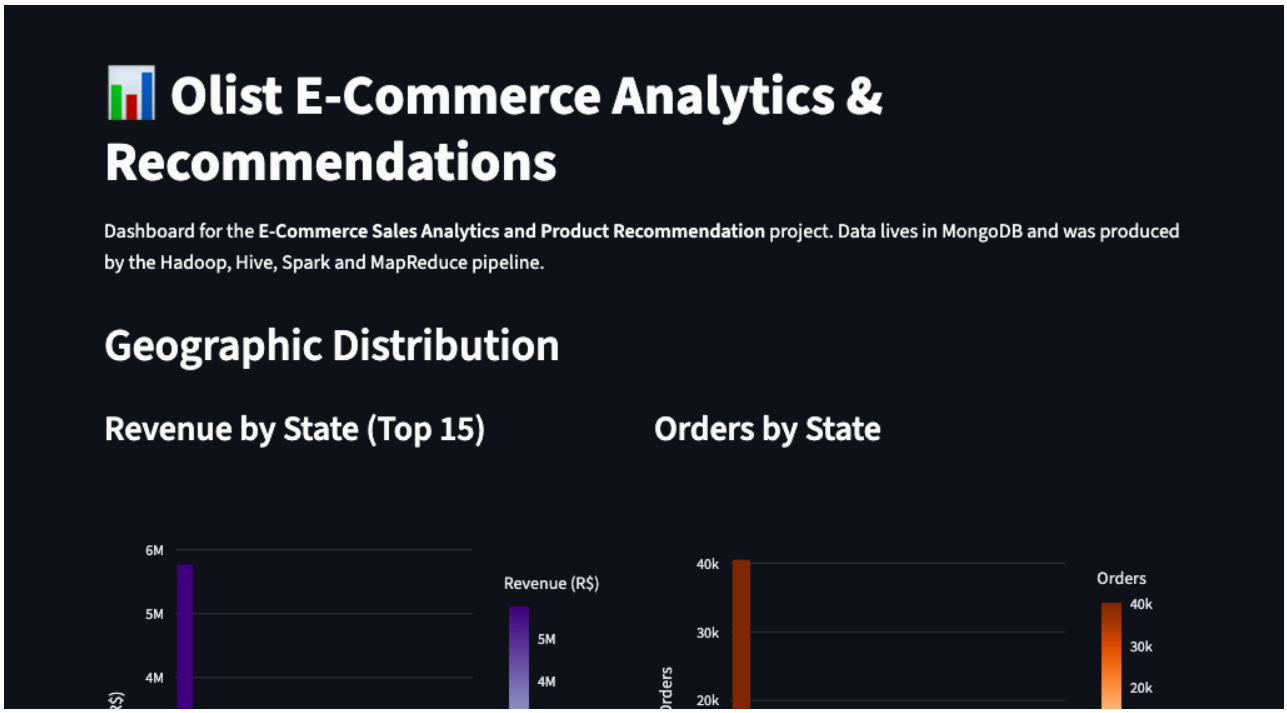


Fig. 7: Geographic distribution of revenue and orders by state.

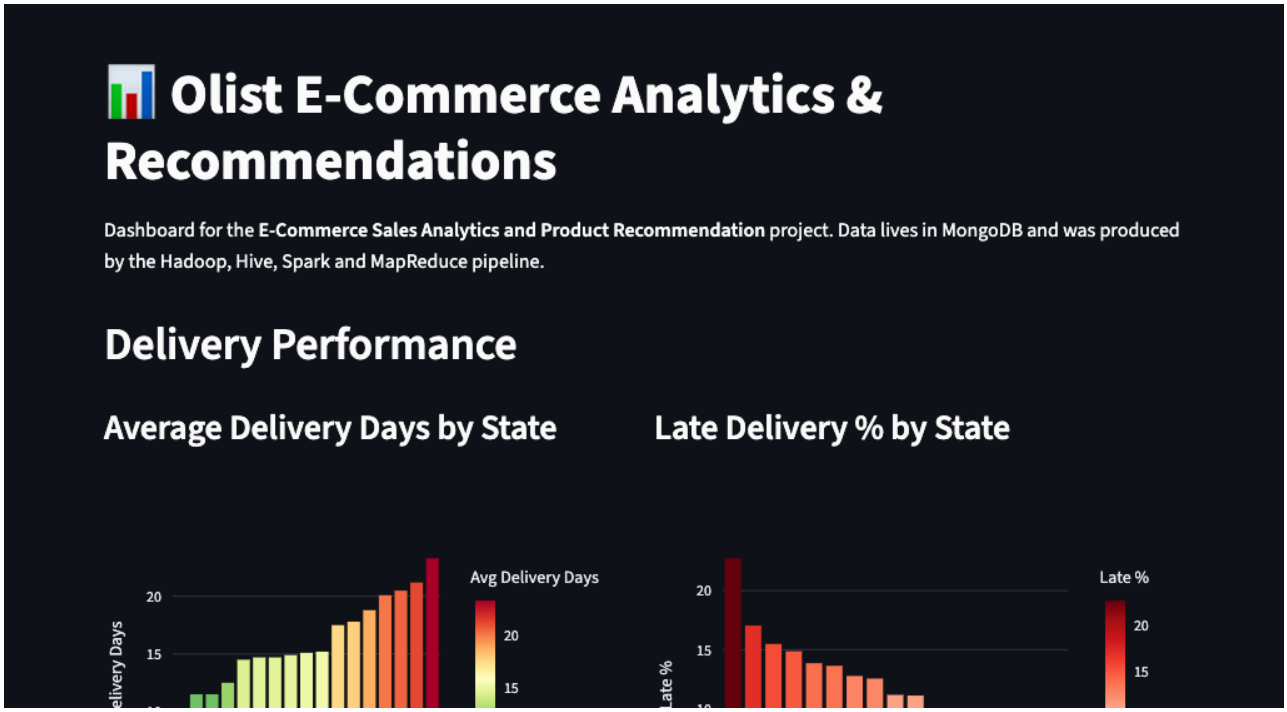


Fig. 8: Delivery performance and review score distribution.

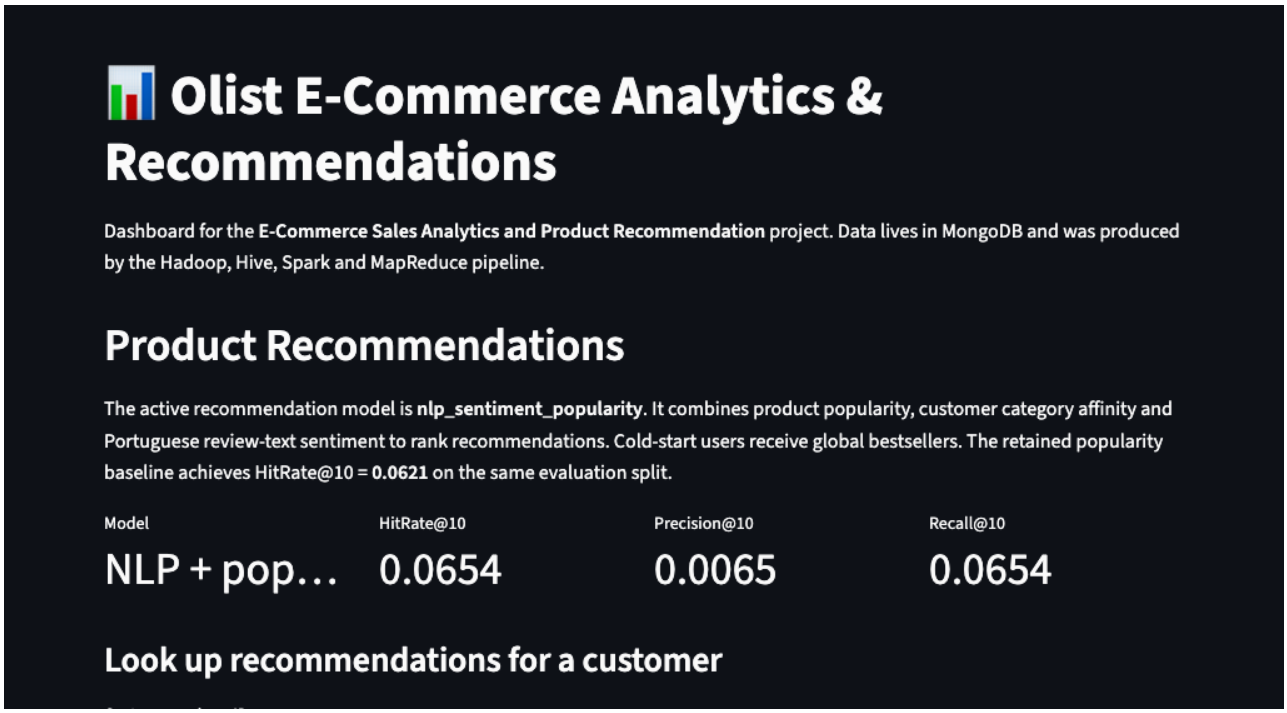


Fig. 9: Recommendations view showing the active NLP model metrics and customer lookup.

6. Limitations and Reproducibility

The following limitations affect interpretation of the results and should be considered when reproducing the work.

- The sentiment lexicon is hand-curated (1,260 tokens) and does not capture domain-specific Portuguese slang or sarcasm. A pretrained model would likely be more robust.
- Reviews are order-level, so product-level sentiment is a revenue-weighted proxy, not direct product satisfaction feedback. A positive order review may reflect satisfaction with the dominant product, the seller or delivery.
- The evaluation uses a same-dataset leave-one-out split with a single precomputed held-out set for both models. Popularity, category lists and sentiment are computed from the training portion only; however, it is still not a time-based train/test split, so the metrics are a ranking benchmark rather than an estimate of future performance.
- The dataset is extremely sparse (≈1.07 purchases per customer), so absolute HitRate@10 values are modest for any model.
- Hive ran in local mode because of a log4j2 bridge conflict with YARN containers; this is a deployment limitation, not a methodology flaw.

Reproducibility: the full pipeline can be executed from a single command (**run_project.sh**) after setting environment variables in **config/settings.sh**. The DA-3 loader and recommender are run with **python3 -m src.recommender.loader_and_recommender**, the dashboard with **python3 -m streamlit run src/dashboard/app.py**, and tests with **python3 tests/test_da3.py**. All generated figures, reports and screenshots are written to **out/report/**.

7. Lessons Learned

- A clean landing layer is essential when source CSVs contain embedded newlines and inconsistent quoting.
- Cross-engine validation (Hive vs Spark) exposes subtle definition differences, such as whether freight is included in revenue.
- Honest reporting of negative results is better than hiding them; ALS's failure motivated the simpler, better baseline.
- Portuguese review text provides a measurable ranking signal, but attribution is a proxy because reviews are order-level, not product-level.
- Confidence weighting prevents sparse review sentiment from dominating the recommendation score.
- JDK 21 strong encapsulation requires explicit --add-opens flags across Hadoop, Hive, Spark and MapReduce task JVMs.
- A document store is a natural fit for per-user recommendation lists and aggregate dashboards.
- Dashboard aggregates should be pre-computed and stored in the serving layer rather than computed at page load.
- Hadoop paths with spaces break URI parsing, so transfers must use relative paths or space-free staging directories.

8. Conclusion and Future Scope

The project demonstrates a complete batch Big Data pipeline over real e-commerce data. All planned components from DA-1 were implemented, including the dashboard that was deferred to DA-3. For DA-3 a Portuguese NLP sentiment

pipeline and a sentiment-aware recommender were added, evaluated against the popularity baseline and integrated into the MongoDB serving layer. The pipeline is reproducible from a single command (run_project.sh) and every number is traceable to a log file.

Key outcomes include: (1) a working pseudo-distributed Hadoop/Hive/Spark/MongoDB stack on a laptop; (2) three MapReduce jobs with combiners and a streaming job; (3) a Hive warehouse with ten analytics queries; (4) Spark ETL and SQL that cross-validate the Hive results; (5) a Portuguese NLP sentiment pipeline that processes 40,950 review texts and attributes sentiment to 18,908 products; (6) an evaluated NLP-aware recommender that improves HitRate@10 by 0.33 percentage points (5.3% relative) over the baseline; and (7) a verified Streamlit dashboard served from MongoDB.

Future extensions include:

- Spark Structured Streaming over the review stream to cover the data-streams module in real time.
- A pretrained Portuguese sentiment model or multilingual transformer when environment constraints allow.
- Explicit product-level review attribution if product review data becomes available.
- A stricter time-based evaluation split to estimate true future recommendation performance.
- Deployment of the dashboard to a container or cloud platform for live demo access.
- Synthetic scale-up of the dataset to test the pipeline at multi-gigabyte size.

References

1. Olist. Brazilian E-Commerce Public Dataset by Olist, version 6. Kaggle, 2018. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce (accessed October 2026).

2. Apache Software Foundation. Apache Hadoop 3.5.0 documentation. https://hadoop.apache.org/docs/r3.5.0/ (accessed October 2026).

3. Apache Software Foundation. Apache Hive 4.2.1 documentation. https://hive.apache.org/ (accessed October 2026).

4. Apache Software Foundation. Apache Spark 4.2.0 documentation. https://spark.apache.org/docs/4.2.0/ (accessed October 2026).

5. MongoDB Inc. MongoDB Community Server 8.3.11 manual. https://www.mongodb.com/docs/manual/ (accessed October 2026).

6. Streamlit Inc. Streamlit 1.50.0 documentation. https://docs.streamlit.io/ (accessed October 2026).

7. Loper, E. and Bird, S. NLTK: The Natural Language Toolkit. Proceedings of the ACL Workshop on Effective Tools and Methodologies for Teaching Natural Language Processing and Computational Linguistics, 2002.